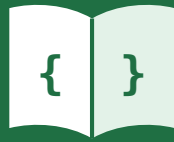

Modül 06 — Spring Data JPA ve Hibernate

Ödevler

Spring Boot Edu



2026-09-23

İçindekiler

Nasıl Çalışılır?	2
Ödev 1 — Sipariş Aggregate'ini Eşlemek (Kolay)	2
Ödev 2 — N+1'i Bulmak ve Düzeltmek (Orta)	3
Ödev 3 — Dinamik Sipariş Arama (Zor)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır?

1. Başlangıç kodu modules/06-data-jpa-postgres/exercise/ içindedir. TODO yorumlarını bulun.
2. Testler gerçek bir PostgreSQL ile çalışır (Testcontainers). **Docker açık olmalıdır.**
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/06-data-jpa-postgres/exercise -am test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takıldığınızda önce ipuçlarına, sonra modules/06-data-jpa-postgres/solution/ içindeki çözüme bakın.

! Önemli

Ödevler sırayla yapılmalıdır. Ödev 2 ve 3, Ödev 1’de eşleyeceğiniz sipariş satırlarına ihtiyaç duyar. Tablolar ve örnek veriler (V1, V2) hazırdır.

Ödev 1 — Sipariş Aggregate’ini Eşlemek (Kolay)

Hedef: İki yönlü bir 1-N ilişkiyi cascade ve orphan removal ile doğru eşlemek.

Bir sipariş (PurchaseOrder) satırlarıyla (OrderLine) birlikte kaydedilmeli ve silinmelidir. Satırlar yalnızca sipariş üzerinden eklenip çıkarılır.

Yapılacaklar (exercisel paketi):

- TODO 1a — PurchaseOrder.lines: @Transient yerine 1-N eşleme. İlişkinin sahibi OrderLine.order’dır. Satırlar siparişle birlikte kaydedilir. Listedden çıkarılan satır veritabanından silinir.
- TODO 1b — OrderLine.order: @Transient yerine order_id sütununa lazy bir N-1 eşleme.
- TODO 1c — addLine ve removeLine: ilişkinin **iki tarafını** birlikte kurun.

İpuçları:

- Ders bölüm 3.1: @OneToMany(mappedBy = ..., cascade = ..., orphanRemoval = ...), @ManyToOne(fetch = FetchType.LAZY), @JoinColumn.
- Yeni satır new OrderLine(this, ...) ile oluşturun.

Kabul kriterleri: Exercise1Test içindeki 4 testin hepsi geçer.

Tahmini süre: 30 dakika

Ödev 2 — N+1'i Bulmak ve Düzeltmek (Orta)

Hedef: Bir N+1 problemini SQL sayısı ile kanıtlamak, düzeltmek ve bir raporu veritabanına hesaplatmak.

`findByCustomerEmail` bir müşterinin siparişlerini getirir. Toplamları hesaplamak için her siparişin satırlarına dokunmak $1 + N$ sorgu çalıştırır. Test bunu ölçer (`theProblem_oneQueryPerOrder` zaten yeşildir).

Yapılacaklar (`exercise2.OrderRepository`):

- TODO 2a — `findWithLinesByCustomerEmail`: siparişleri **ve** satırlarını tek bir SQL ile yükleyin.
- TODO 2b — `totalsPerCustomer`: varsayılan metodu, her müşteri için bir `CustomerTotal` döndüren bir JPQL sorgusuyla değiştirin. Toplam = tüm satırlarda `quantity * unitPrice`. En yüksek toplam önce gelmeli ve hesabı veritabanı yapmalı.

İpuçları:

- Ders bölüm 3.3: `@EntityGraph(attributePaths = ...)`.
- Ders bölüm 3.4: `select new paket.CustomerTotal(...) ... group by ... order by ....`

Kabul kriterleri: `Exercise2Test` içindeki 3 testin hepsi geçer (her iki çözüm de **tek** SQL).

Tahmini süre: 30 dakika

Ödev 3 — Dinamik Sipariş Arama (Zor)

Hedef: Opsiyonel kriterlerden `Specification` ile esnek bir sorgu kurmak.

Yapılacaklar (`exercise3` paketi):

- TODO 3a — `OrderSpecifications`: müşteri, durum ve “şu tarihten sonra oluşturulan” kriterleri.
- TODO 3b — `containsIsbn`: siparişin bu ISBN’e sahip en az bir satırı olsun. Her sipariş sonuçta **bir kez** görünsün.
- TODO 3c — `OrderSearch.search`: yalnızca dolu kriterleri birleştirin (hepsi sağlanmalı), sayfalı sorgulayın ve her siparişi bir `OrderSummary`’ye çevirin.

İpuçları:

- Ders bölüm 3.5: `(root, query, cb) -> cb.equal(root.get("..."), ...), cb.greaterThan, root.join("lines")`.
- Bir koleksiyona `join` yapmak aynı siparişi birden çok kez döndürebilir: `query.distinct(true)`.

- `Specification.allOf(liste)` ve `repository.findAll(spec, pageable)`.

Kabul kriterleri: Exercise3Test içindeki 5 testin hepsi geçer.

Tahmini süre: 45 dakika

Ek Meydan Okuma (İsteğe Bağlı)

PurchaseOrder'a @Version ekleyin (yeni bir Flyway migration'ı ile version sütunu). İki eş-zamanlı güncellemeden birinin `ObjectOptimisticLockingFailureException` ile reddedildiğini gösteren bir test yazın (ders bölüm 3.6).